

Q1. Explain Linear and Non Linear Data structure in Python with examples. [3 Marks]

1. Linear Data Structure:

- **Definition:** A data structure in which elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements.
- **Characteristics:** Elements can be traversed in a single run (single level). Memory is allocated sequentially or via pointers.
- **Examples:**
 - **List / Array:** numbers = [10, 20, 30]
 - **Stack:** Follows LIFO (Last In First Out).
 - **Queue:** Follows FIFO (First In First Out).
 - **Linked List:** Nodes connected via pointers.

2. Non-Linear Data Structure:

- **Definition:** A data structure where data elements are not arranged sequentially; instead, elements are arranged in a hierarchical or interconnected manner across multiple levels.
- **Characteristics:** Elements cannot be traversed in a single pass.
- **Examples:**
 - **Tree:** Hierarchical structure with a root and child nodes (e.g., Binary Search Tree).
 - **Graph:** A collection of vertices (nodes) and edges connecting them (e.g., Network models).

Q2. Define following terms regarding Tree data structure: [5 Marks]

1. In-degree:

- a. It is defined as the total number of incoming edges pointing directly to a specific node.
- b. In any tree, the **root node** has an in-degree of **\$0\$**, while every other node has an in-degree of **\$1\$**.

2. Out-degree:

- a. It is defined as the total number of outgoing edges originating from a node pointing to its children.

- b. It is equal to the total number of children a node possesses. A leaf node always has an out-degree of **0**.

3. Depth:

- a. The depth of a node is the length of the path (i.e., the number of edges) from the root node to that specific node.
- b. The depth of the root node is always **0**.

Q3. Explain different operations on stack with any one algorithm. [5 Marks]

Stack Principle: Follows **LIFO** (Last In First Out).

Primary Operations on Stack:

- **push(item):** Adds a new element to the top of the stack.
- **pop():** Removes and returns the topmost element from the stack.
- **peek() / top():** Returns the topmost element without removing it.
- **isEmpty():** Checks whether the stack contains zero elements.
- **isFull():** Checks if the stack has reached its maximum defined capacity (Overflow condition).

Algorithm: PUSH Operation (Using Python List)

- **Step 1:** Start.
- **Step 2:** Check for Stack Overflow:
 - If `len(stack) == MAX_SIZE`, print "Stack Overflow" and exit.
- **Step 3:** Input the new element `item`.
- **Step 4:** Insert the element at the top using:

`stack.append(item)`

- **Step 5:** Print "Item pushed successfully".
- **Step 6:** Stop.

Q3 (OR). Explain different operations on queue with any one algorithm. [5 Marks]

Queue Principle: Follows **FIFO** (First In First Out).

Primary Operations on Queue:

- **enqueue(item):** Inserts an element at the rear (end) of the queue.
- **dequeue():** Deletes and returns the element from the front of the queue.
- **front() / peek():** Returns the element at the front without deleting it.
- **isEmpty():** Returns True if the queue has no elements.
- **isFull():** Returns True if the queue has reached its maximum size.

Algorithm: ENQUEUE Operation (Using Python List)

- **Step 1:** Start.
- **Step 2:** Check for Queue Overflow:
 - If `len(queue) == MAX_SIZE`, print "Queue Overflow" and exit.
- **Step 3:** Input the new element `item`.
- **Step 4:** Add element to the rear of the queue:

`queue.append(item)`

- **Step 5:** Print "Item enqueued successfully".
- **Step 6:** Stop.

Q4. Define List. Write various operations that can be performed on List. [3 Marks]

Definition:

A **List** in Python is an ordered, mutable (changeable), and dynamic sequence data structure that can store heterogeneous (mixed data types) elements enclosed inside square brackets `[]`.

Various Operations on List:

1. **Append:** Adds an element at the very end.

Python

```
l = [1, 2]
l.append(3)  # [1, 2, 3]
```

2. **Insert:** Adds an element at a specific index.

Python

```
l.insert(1, 10)  # [1, 10, 2, 3]
```

3. **Pop:** Removes and returns the element at the specified position (default is last).

Python

```
val = l.pop() # removes 3
```

4. **Remove:** Deletes the first occurrence of a specific value.

Python

```
l.remove(10) # removes 10
```

5. **Slicing / Indexing:** Accesses sub-parts or individual elements.

Python

```
x = l[0] # returns 1
```

Q5. Convert following algebraic expression into postfix notation using Stack. [5 Marks]

Sub-Question (ii): $a - b / (c * d / e)$

Operator Precedence: $^ > *, / > +, -$ (Left to Right associativity for $*, /, +, -$).

Symbol Read	Stack	Postfix Expression Output
a	(empty)	a
-	-	a
b	-	a b
/	- /	a b
(	- / (	a b
c	- / (	a b c
*	- / (*	a b c
d	- / (*	a b c d

/	- / (/ (pop *)	a b c d *
e	- / (/	a b c d * e
)	- / (pop /)	a b c d * e /
End	(pop /, then -)	a b c d * e / / -

Final Postfix (ii):

$\mathbf{a\ b\ c\ d\ *\ e\ /\ /\ -}$

Sub-Question (i): $(a * b) * (c \text{ } d * (d + e) - f)$

(Assuming missing standard multiplication operator between d and $(d+e)$)

Symbol Read	Stack	Postfix Expression Output
(	(	(empty)
a	(	a
*	(*	a
b	(*	a b
)	(empty, pop *)	a b *
*	*	a b *
(	* (	a b *
c	* (	a b * c
^	* (^	a b * c
d	* (^	a b * c d
*	* (* (pop ^)	a b * c d ^

(	* (* (	a b * c d ^
d	* (* (	a b * c d ^ d
+	* (* (+	a b * c d ^ d
e	* (* (+	a b * c d ^ d e
)	* (* (pop +)	a b * c d ^ d e +
-	* (- (pop *)	a b * c d ^ d e + *
f	* (-	a b * c d ^ d e + * f
)	* (pop -)	a b * c d ^ d e + * f -
End	(empty, pop *)	a b * c d ^ d e + * f - *

Final Postfix (i):

$$\mathbf{a\ b\ *\ c\ d\ \text{\textasciicircum}\ d\ e\ +\ *\ f\ -\ *}$$

Q6. Draw Binary search tree for 8, 4, 12, 2, 6, 10, 14, 1, 3, 5 and write In-order traversal. [5 Marks]

BST Property: Left child $\leq$ Root $\leq$ Right child.

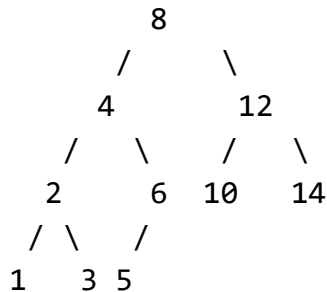
Step-by-step Insertion:

1. Insert 8 $\rightarrow$ Root
2. Insert 4 $\rightarrow$ Left of 8
3. Insert 12 $\rightarrow$ Right of 8
4. Insert 2 $\rightarrow$ Left of 4
5. Insert 6 $\rightarrow$ Right of 4
6. Insert 10 $\rightarrow$ Left of 12
7. Insert 14 $\rightarrow$ Right of 12
8. Insert 1 $\rightarrow$ Left of 2
9. Insert 3 $\rightarrow$ Right of 2

10. Insert 5 $\rightarrow$ Left of 6

BST Diagram to Draw:

Plaintext



In-order Traversal (Left $\rightarrow$ Root $\rightarrow$ Right):

$\mathbf{\{1, 2, 3, 4, 5, 6, 8, 10, 12, 14\}}$

(Note: In-order traversal of any valid BST always yields elements in strictly ascending sorted order)

Q6 (OR). Draw Binary search tree for 8, 4, 12, 2, 6, 10, 14, 1, 3, 5 and write post-order traversal. [5 Marks]

(Tree diagram remains identical to above)

Post-order Traversal (Left $\rightarrow$ Right $\rightarrow$ Root):

1. Left subtree of 8:
 - a. Left subtree of 4: Left of 2 is 1, Right of 2 is 3, Root is 2 $\rightarrow$ 1, 3, 2
 - b. Right subtree of 4: Left of 6 is 5, Root is 6 $\rightarrow$ 5, 6
 - c. Subtree 4 root $\rightarrow$ 4
 - d. Combined left side: 1, 3, 2, 5, 6, 4
2. Right subtree of 8:
 - a. Left of 12 is 10, Right of 12 is 14, Root is 12 $\rightarrow$ 10, 14, 12
3. Overall Root: 8

Post-order Result:

$\mathbf{\{1, 3, 2, 5, 6, 4, 10, 14, 12, 8\}}$

Q7. Write a Python class to represent a Cricketer. [4 Marks][cite: 1, 2]

Python

```
class Cricketer:
    # 1. Member function to initialize data members
    def __init__(self, name, team_name, run=0):
        self.name = name          # cricketer name
        self.team_name = team_name # team name
        self.run = run            # runs scored

    # 2. Member function to set runs
    def set_run(self, run):
        self.run = run

    # 3. Member function to display details
    def display_run(self):
        print("Cricketer Name :", self.name)
        print("Team Name      :", self.team_name)
        print("Runs Scored    :", self.run)

# --- Demonstration Code ---
c1 = Cricketer("Virat Kohli", "India", 82)
c1.display_run()

print("\nUpdating Runs...")
c1.set_run(117)
c1.display_run()
```

Q7 (OR). Explain concept of recursive function with any one suitable code in python. [4 Marks]**Concept of Recursive Function:**

- A function that calls itself directly or indirectly to solve a smaller instance of the same problem is called a **Recursive Function**.
- Every recursive function must have two mandatory components:
 - **Base Case:** The termination condition that stops recursion and prevents infinite loops.

- **Recursive Step:** The part where the function calls itself with modified/reduced parameters, moving closer towards the base case.

Example: Factorial of a Number

Mathematical formula:

$$n! = n \times (n-1)! \quad (\text{Base condition: } 0! = 1 \text{ or } 1! = 1)$$

Python

```
def factorial(n):
    # Base Case
    if n == 0 or n == 1:
        return 1
    # Recursive Case
    else:
        return n * factorial(n - 1)

# Driver Code
num = 5
result = factorial(num)
print(f"The factorial of {num} is: {result}") # Output: 120
```